

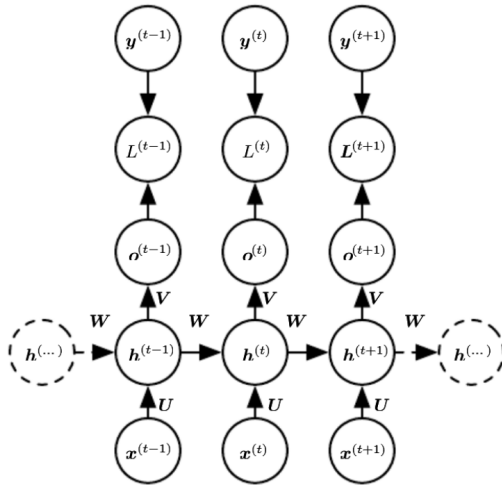
Deep Learning — Final Exam

Date: Thursday, May 29, 2025

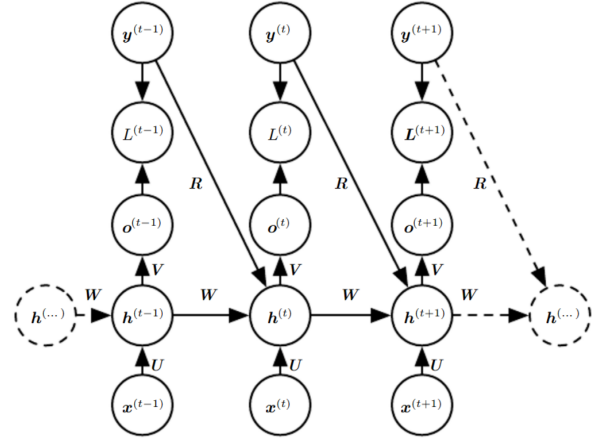
Time: 12:20 – 15:10 (170 minutes)

Instructions:

- You may give your answers in Chinese or English.
- Please give your answers in succinct phrases or point form.
- Please write your answers clearly (with explicit denotation of labels and symbols used).
- Download the exam sheet from the materials section before the exam begins. The file will be removed from NTUCOOL 20 minutes after the exam starts, and will no longer be accessible.
- You must upload your answer sheet before leaving the classroom. After uploading, please be sure to complete the sign-out procedure. Do not upload or re-upload your answers after leaving the room. If we detect that an answer sheet was submitted after sign-out, or that the student did not sign out properly, the uploaded answer will be considered invalid and will receive a score of zero.



(a) Design Pattern I



(b) Design Pattern II

1. (10 pts) Recurrent Neural Networks (RNNs).

- (a) (2 pts) Explain why the design pattern I of RNN implies that the model outputs $o_1, o_2, \dots, o_n$ are conditionally independent at test time given the input sequence $x_1, x_2, \dots, x_n$.
- (b) (2 pts) Explain in what sense the design pattern II is more general than pattern I.
- (c) (3 pts) In training a RNN with the design pattern II, one may need to apply the technique *teacher forcing*. Explain the idea.

- (d) (3 pts) What issues do you expect may arise if in the output recursion, one always chooses the ground truth $y_1, y_2, \dots, y_n$ for training, or the previous model outputs $o_1, o_2, \dots, o_n$ as substitutes for $y_1, y_2, \dots, y_n$.
2. (35 pts) Please compare the following generative models: (1) the RNN-based auto-regressive model, (2) variational autoencoder (VAE), (3) generative adversarial network (GAN), (4) flow model, and (5) diffusion model.
- (a) (5 pts) Which models would allow an exact evaluation of the likelihood of a data sample?
 - (b) (5 pts) Specify the training objective for each of these models, e.g. maximizing the data likelihood, evidence lower bound, or JS-divergence.
 - (c) (5 pts) In terms of the data generation process, please describe how to draw samples from each of these models. Be specific about whether the generator generates data samples or data distributions.
 - (d) (5 pts) What are the major advantages of the diffusion model as compared to GAN?
 - (e) (10 pts) In designing a diffusion model, a very small Gaussian noise is introduced in every forward diffusion step. That is, $x_t = \sqrt{1 - \beta_t}x_{t-1} + N$, where N is a Gaussian noise with mean zero and a small variance β_t . Please explain why β_t has to be small in order to ensure that the inverse denoising process $p_\theta(x_{t-1}|x_t)$ follows a conditional Gaussian distribution.
 - (f) (5 pts) What are the major differences between GAN and W-GAN in terms of their training objectives? Does the critic in W-GAN play the same role as the discriminator in GAN?
3. (20 pts) Training the VAE.
- (a) (4 pts) In training the VAE, we try to maximize a variational lower bound on the data log-likelihood. Explain the main idea and provide the exact objective function to be maximized.
 - (b) (4 pts) What distribution does the approximate posterior $q(z|x)$ take for training VAE? Is this an assumption?
 - (c) (4 pts) Explain the notion of the re-parameterization trick.
 - (d) (4 pts) True or False: In maximizing the variational lower bound, the approximate posterior $q(z|x)$ should ideally be identical to the prior $p(z)$ when the variational lower bound is maximized. Explain your answer.
 - (e) (4 pts) How would you evaluate the KL divergence $KL(q(z|x)||p(z))$ if the prior $p(z)$ is replaced with a Gaussian Mixture distribution?

4. (6 pts) Activation Functions

Describe the advantages of using the ReLU activation function in deep neural networks. Then, explain one limitation of ReLU and how Leaky ReLU addresses this issue.

5. (6 pts) CNN Calculation

An image of size $64 \times 64 \times 3$ is passed through the following layers sequentially:

- Conv layer: 32 filters of size 3×3 , stride 1, padding 1
- ReLU activation
- Max pooling: 2×2 , stride 2, no padding

- (2pts) What is the output size after the convolution layer?
- (2pts) How many learnable parameters are there in the convolution layer?
- (2pts) What is the final output size after the pooling layer?

6. (7 pts) Transformer - Attention Weight Computation

Given a Transformer input sequence with 4 tokens, each with embedding dimension $d=4$. The query and key matrices for a head are as follows:

- Query vector for token A: $[1, 0, 1, 0]$
- Key vectors for tokens A, B, C, D:
 - A: $[1, 0, 1, 0]$
 - B: $[0, 1, 0, 1]$
 - C: $[1, 1, 0, 0]$
 - D: $[0, 0, 1, 1]$

- (3pts) Compute the dot product attention scores for token A with all tokens.
- (4pts) Apply softmax to get attention weights (you can leave the answers as expressions like $\exp(x)/Z$).

7. (6 pts) Vision Transformer Patching

An RGB image of size $224 \times 224 \times 3$ is processed by a Vision Transformer with a patch size of 16×16 .

- (2pts) How many patches does the image get divided into?
- (2pts) What is the dimension of each flattened patch vector?
- (2pts) What is the input size (rows $\times$ columns) of the sequence fed to the Transformer encoder after patch embedding and [CLS] token insertion?

8. (15 pts) In Lecture 11, we have discussed several policy-based RL algorithms that are built on policy gradient, including REINFORCE, Advantage Actor-Critic (A2C), and PPO with a clipped surrogate objective (PPO-Clip):

- (4pt) Among the policy gradient expressions, REINFORCE policy gradient offers a useful expression for improving the policy:

$$\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \mathbb{E}_{\tau \sim P_{\mu}^{\pi_{\theta}}} \left[\sum_{t=0}^{\infty} \gamma^t Q^{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (1)$$

Based on (1), please write down the pseudo code of the classic REINFORCE algorithm, where we only maintain the policy network without learning a critic. (**Hint:** You shall explain how we can estimate the policy gradient in (1) based on the sampled trajectories)

- (3pt) An alternative REINFORCE policy gradient expression is

$$\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \mathbb{E}_{\tau \sim P_{\mu}^{\pi_{\theta}}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right], \quad (2)$$

where $A^{\pi_{\theta}}(s, a) := Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s)$ is the advantage function. What is the purpose of introducing $A^{\pi_{\theta}}(s, a)$ in (2)?

- (4pt) Built on the policy gradient expression in (2), the A2C algorithm maintains two networks, *i.e.*, an actor network π_{θ} (or a policy network) and a critic network (either $Q_w(s, a) \approx Q^{\pi_{\theta}}(s, a)$ or $V_w(s) \approx V^{\pi_{\theta}}(s)$), which is used to estimate the advantage function $A^{\pi_{\theta}}(s, a)$. Can you design a way to learn the critic under value function approximation and estimate $A^{\pi_{\theta}}(s, a)$? Please clearly specify the critic network, the loss function, and how $A^{\pi_{\theta}}(s, a)$ is estimated (**Hint:** For example, you may use TD learning and GAE)
- (4pt) PPO-Clip can be viewed as an enhanced variant of A2C. Specifically, the objective function of PPO-Clip is

$$\mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta_k}}, a \sim \pi_{\theta_k}(\cdot | s)} \left[\min \left\{ \frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)} A^{\theta_k}(s, a), \text{clip}\left(\frac{\pi_{\theta}(a | s)}{\pi_{\theta_k}(a | s)}, 1 - \varepsilon, 1 + \varepsilon\right) A^{\theta_k}(s, a) \right\} \right] \quad (3)$$

Can you explain the purpose of adding the “clip” operation in (3)? Moreover, why do we need an additional “min” operator in (3)?

9. (10 pts) Double DQN is a widely-used value-based algorithm in the family of DQNs. Algorithm 1 below shows the draft pseudo code of Double DQN provided by ChatGPT.
- (4pt) There are mistakes in the pseudo code. Please identify these mistakes and make the corresponding corrections. Please write your responses in the form of “Line Number: Describe the mistake. Provide the correct implementation.”
 - (3pt) To apply Double DQN to continuous control, apart from action discretization, can you provide one technique that can adapt Double DQN to continuous action spaces?
 - (3pt) To further improve Double DQN, can you name 2 possible enhancements (e.g., the loss function, the replay buffer, or the exploration strategy)?

Algorithm 1 Double DQN

```

1: Initialize Q-network with weights  $\theta$ 
2: Initialize target network  $Q'$  with weights  $\theta' = \theta$ 
3: Initialize replay buffer  $\mathcal{D}$  and discount factor  $\gamma \in [0, 1)$ 
4: for episode = 1 to  $M$  do
5:   Initialize state  $s_0$ 
6:   for  $t = 0$  to  $T$  do
7:     With probability  $\varepsilon$ , select a random action as  $a_t$ ; Otherwise,  $a_t \leftarrow \arg \max_a Q(s, a; \theta)$ 
8:     Execute action  $a_t$ , observe reward  $r_{t+1}$  and next state  $s_{t+1}$ 
9:     Store transition  $(s_t, a_t, r_{t+1}, s_{t+1})$  in  $\mathcal{D}$ 
10:    Sample a minibatch  $B \equiv \{(s, a, r, s')\}$  from  $\mathcal{D}$ 
11:    Reset the loss  $L_\theta = 0$ 
12:    for each transition  $(s, a, r, s')$  in the minibatch  $B$  do
13:      if  $s'$  is terminal then
14:         $y \leftarrow 0$ 
15:      else
16:         $y \leftarrow r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta'); \theta)$ 
17:      end if
18:       $L_\theta \leftarrow L_\theta + (y - Q(s, a; \theta'))^2$ 
19:    end for
20:    Perform gradient descent on  $L_\theta$  to update  $\theta$ 
21:    Update target network:  $\theta' \leftarrow \theta$ 
22:  end for
23: end for

```
